Modulo di orchestrazione conversazionale (sviluppo UniFI MICC)

Questa nuova fase descrive il componente software — realizzato dal laboratorio MICC dell'Università di Firenze — che collega il chatbot agli **MCP tool** esposti dal server.

Il modulo, scritto in Python e basato sul framework **Agno**, traduce l'intento rilevato nei messaggi dell'utente in una chiamata MCP correttamente parametrizzata, gestendo in autonomia la raccolta e la validazione dei dati mancanti.

Nella tabella 1 si descrivono le tecnologie utilizzate con relativi task e motivazioni.

Tecnologie impiegate	**Scelta**	**Motivazione**
Framework multi-agente	Agno	Offre astrazione "Agent / Team" già pronta per orchestrare più specialisti.
LLM / Embedding	OpenAI gpt-4o-mini (chat) • OpenAIEmbedding	Modelli compatti con ottimo trade-off costo/qualità; embedding unificato per la Knowledge Base.
Vector DB	LanceDB	Storage colonnare ottimizzato per embedding, facile da versionare.
Knowledge Base	JSONKnowledgeBase	Carica gli intenti dichiarativi già prodotti nella Fase 1 (file JSON).
Memoria conversazionale	Agno Memory v2 + SQLite	Persiste il contesto di lungo periodo e i parametri già confermati.

Tabella 1

Architettura logica a più agenti

Il framework **Agno** consente di strutturare una soluzione conversazionale secondo un paradigma *multi-agente*, suddividendo le diverse responsabilità operative in moduli autonomi ma coordinati. Ciascun **Agente** è definito da:

1. **Ruolo funzionale** – ad esempio *Intent Recognizer* o *API Caller*.

2. **Prompt di servizio** – un insieme di istruzioni che ne regolano il comportamento linguistico e procedurale.

3. **Risorse di supporto** – memoria conversazionale, knowledge-base vettoriale, collegamenti a servizi esterni.

Gli agenti sono riuniti in un **Team**; il suo nodo apicale è l'**Agente Coordinatore**, che opera come regista del flusso.

Il ciclo di lavoro si avvia con il messaggio dell'utente, trasmesso dal componente di chat integrato nel sito. Il Coordinatore:

- riceve l'input;

- determina quale agente debba gestire la fase corrente (riconoscimento dell'intento, raccolta parametri, invocazione API, ecc.);

- aggrega e normalizza le risposte intermedie;

- garantisce coerenza semantica e uniformità di tono per tutta la durata dello scambio.

In tal modo, la complessità tecnica rimane confinata all'interno dei singoli agenti, mentre l'utente percepisce un'interazione continua, precisa e priva di riferimenti alla logica sottostante.

In definitiva tutto parte dal messaggio dell'utente attraverso un componente di chat attivato sul sito web di riferimento. L'agente Coordinatore, che funge da regista, lo riceve per primo. Il suo compito è capire a chi passare il task e controllare che l'intero scambio rimanga coerente e dal tono naturale.

Step 1: Intent Recognizer

Il Coordinatore inoltra la frase all'Intent Recognizer: l'agente allenato a riconoscere gli "intenti" presenti nella knowledge base. Se trova una corrispondenza, restituisce un piccolo pacchetto JSON:

```
None
{
```

```
  "intent": "create_invoice",
  "parameters": {"amount": 180, "client_id": "65", "source": "external_db"},
  "missing": {"date": ""}
}
```

Se invece non capisce l'intento (intent: null), il Coordinatore chiede gentilmente all'utente di riformulare la richiesta.

Step 2: Parameter Resolver

Quando nel campo missing c'è qualcosa, entra in gioco il Parameter Resolver. Questo agente si comporta come un assistente "puntiglioso": fa domande, controlla la risposta, integra e, se è valida, aggiorna il JSON. Così facendo evita di sommergere l'utente con richieste multiple.

Se un parametro dipende da un database esterno e il valore è ambiguo («Rossi» può essere Mario o Maria), l'agente mostra le possibili opzioni e chiede di scegliere.

Quando l'ultimo campo viene riempito, il pacchetto è completo e il *turn-around* del Parameter Resolver finisce.

Step: 3: API Caller

L'aPI caller è l'agente in carico di comunicare con l'MPC server per il recupero di informazioni presenti su Database e servizi esterni e di finalizzare l'intento dell'utente.

Riconosciuto l'intento e ottenuti tutti i parametri della richiesta, il Coordinatore passa tutto all'API Caller. Quest'ultimo traduce il JSON in una vera e propria chiamata al tool MCP indicato (ad es. /create-invoice) e aspetta la risposta del server.

E se non c'è un intento preciso? – Chat Memory (facoltativa)

Capita che una richiesta sia più conversazionale («Come stai?» o «Ricordi la fattura di ieri?»). In questi casi, se nessun intento emerge, il Coordinatore delega la risposta all'agente Chat Memory, che sfrutta la cronologia per mantenere continuità e tono amichevole.

Le responsabilità e i ruoli dei vari agenti sono illustrate in Tabella 2.

Agente	Ruolo	Input principale	Output / Azione
Intent Recognizer	Classificare la richiesta e popolare i primi parametri.	Utterance utente + KB	`{"intent": "...", "parameters": {...}, "missing": {...}}`
Parameter Resolver	Validare / completare i parametri mancanti tramite dialogo iterativo.	Struttura JSON dell'agente precedente	Chiedere 1 parametro alla volta; al termine JSON completo.
API Caller	Mappare l'intento sul *tool* MCP e invocarlo.	Intent + parametri validi	Chiamata MCP REST / funzioni; ritorno al chatbot.
(facolt.) Chat Memory	Sfruttare la cronologia per risposte discorsive quando non c'è un intento.	Conversazione	Risposta naturale.
Coordinator (Team)	Smistare il flusso tra gli agenti, gestire l'errore, mantenere il tono.	Messaggi utente e output degli altri agenti	Replica all'utente.

Tabella 2

Gestione dei parametri provenienti da database esterni

In molti processi aziendali i dati necessari per completare un'operazione (clienti, prodotti, codici fiscali, sedi, ecc.) risiedono già in archivi gestionali esistenti e non dovrebbero essere reinseriti manualmente durante la conversazione.

Per questa ragione, nel modello di intenti abbiamo previsto che alcuni campi possano dichiarare "source": "external-db": un'indicazione formale che il valore va recuperato – o quantomeno validato – interfacciandosi a un database di riferimento. Tale meccanismo consente di:

- ridurre errori di digitazione e incongruenze;
- velocizzare la compilazione automatizzando i campi correlati quando la corrispondenza è univoca;
- gestire in modo controllato i casi di ambiguità, chiedendo all'utente solo le informazioni strettamente necessarie;
- garantire che le API MCP ricevano parametri completi, normalizzati e conformi ai vincoli di formato.

Le regole operative che disciplinano questo flusso sono riassunte nel prompt **`gestione_db`** e applicate dall'agente Parameter Resolver durante la raccolta interattiva dei dati.

Quando nel file JSON degli intenti un campo è marcato così:

```
None
"client_name": {
  "type": "str",
  "description": "cliente",
  "source": "external-db",
  "entity": "customer"
}
```

significa che quel valore non deve essere digitato a mano, bensì recuperato – o almeno validato – consultando la tabella *customer* di un database aziendale.

Il prompt **`gestione_db`**, incluso nei nostri agenti, stabilisce le regole di comportamento in quattro punti chiave.

1. Ambiguità: quando il nome non basta

Se l'utente scrive *«Rossi»* e nel database esistono più anagrafiche con quel cognome, l'assistente **non** indovina.
Mostra invece l'elenco delle opzioni ("Mario Rossi", "Maria Rossi", "Rosanna Rossi …") e sospende qualunque compilazione automatica finché l'utente non sceglie. In questo modo si evita di intestare una fattura al destinatario sbagliato o, peggio, di riempire campi correlati (ID cliente, indirizzo, partita IVA) con dati di un omonimo.

2. Corrispondenza univoca: riempimento automatico dei campi collegati

Se la ricerca restituisce **una sola** riga, l'assistente assume che sia quella giusta e completa in blocco tutti i parametri che condividono la stessa `entity`.
Esempio: l'utente scrive *«Ditta Alfa»*; il sistema trova l'unico record corrispondente e popola da sé `clientId`, `indirizzo`, `partita_iva`, lasciando all'utente solo gli elementi realmente mancanti (per esempio la data di scadenza della fattura).

3. Convalide tipizzate: formati obbligatori e controlli di coerenza

- **Date**: devono uscire in formato ISO `YYYY-MM-DD`. Se l'utente usa espressioni colloquiali – *«fine mese», «questa settimana»* – l'assistente le traduce; *«fine mese»* diventa `2025-06-30`, *«questa settimana»* si scompone in `start_date` e `end_date` calcolati rispetto alla data corrente.

- **Numeri**: importi e quantità sono accettati solo se interpretabili come `float`/`int`.

- **Stringhe**: non devono essere vuote né composte solo da spazi.

4. Valore inesistente: invito a correggere o inserire

Quando la ricerca nel database non produce alcun risultato, l'assistente lo dichiara esplicitamente e chiede come procedere:

«Non ho trovato alcun cliente chiamato "Dittta Alfa". Vuoi riprovare con un altro nome o aggiungerne uno nuovo?»

Il flusso, insomma, non prosegue finché l'utente non fornisce un riferimento valido.

Caso d'uso "creazione fattura"

Utente: «Crea una fattura da 180 € per Rossi, scadenza fine mese.»

Sistema:

1. Cerca "Rossi" ➜ due record → chiede di scegliere.

2. L’utente indica “Mario Rossi”.

3. L’assistente recupera automaticamente `clientId`, indirizzo e partita IVA.

4. Converte *«fine mese»* in `2025-06-30`.

5. Passa il pacchetto completo all’API: la fattura viene creata senza dover porre altre domande oppure deve chiedere conferma all'utente.

Riconoscimento dell’intento e invocazione del *tool* MCP

Per ogni intento definito nella knowledge-base è già indicato **quale funzione MCP dev’essere chiamata**.
Questa informazione è custodita nella chiave `target-tool` del relativo blocco JSON. Ad esempio:

```
None
"create_invoice": {

  …

  "target-tool": "/create-invoice"

}
```

Quando l’agente **API Caller** riceve dal Team Agent l’intento e l’elenco dei parametri validati, procede in tre passaggi:

1. **Costruzione del payload**
   Crea un oggetto JSON con la struttura richiesta dal server MCP (`tool_calls`, `id`, `function.name`, `function.arguments`, …).
2. **Trasmissione sicura**
   Invia il payload via HTTPS.

In questo modo la logica di mappatura resta **interamente dichiarativa**: aggiungere un nuovo intento implica solo aggiornare il file JSON, senza modificare il codice Python dell’agente.

Esempio completo scenario di “creazione fattura”

Passo	Dialogo / Operazione interna
1. Utente	«Crea una fattura da **180 €** per **Rossi**.»
2. Intent Recognizer	Identifica `create_invoice` e popola i parametri: • amount = 180 • client_name = «Rossi» Segnala che manca `date`.
3. Parameter Resolver	Chiede: «Inserisci la data di scadenza.»
4. Utente	«30 giugno.» → l’agente converte in `2025-06-30` e completa i parametri.
5. API Caller	Chiama il *tool* **/create-invoice** con il pacchetto: `{amount: 180, clientId: 9876, date: "2025-06-30" …}`
6. MCP → Coordinator → Utente	Risposta MCP: `{invoice_id: "INV-2025-0470", …}` Messaggio finale: «Fattura creata: **INV-2025-0470**, 180 €, data: 30 giugno 2025.». Confermi?

Dunque, partendo da una richiesta libera in linguaggio naturale, il sistema attraversi tutte le fasi—riconoscimento, completamento parametri, chiamata API—restituendo un risultato puntuale senza mai esporre la complessità interna all’utente.